INSTALL TUTORIAL

Molecular Studio — Installation Guide

From installing GROMACS to building and launching the application, step by step

> Version: Molecular Studio 0.3 (engine catalogue: GROMACS 2025.2)
> Platform: Ubuntu 20.04+ / Debian / Kali / similar Linux distributions
> Reference archive: molecular-studio-v0.3.zip

Contents

1. Overview and installation requirements.....

1. Overview and installation requirements.....1

2. Install the base system packages.....2

3. Install GROMACS.....2

3.1 Route one: from the distribution repositories (recommended).....3

3.2 Route two: through Conda.....3

3.3 Route three: build from source (newest release).....4

4. Install the build toolchain: Rust, then Qt6.....5

4.1 Install Rust (builds the application core).....5

4.2 Install Qt6 (builds the graphical interface).....6

5. Build the project.....7

5.1 Build the Rust core.....7

5.2 Build the Qt interface.....8

6. Run the application and verify the installation.....9

7. Important environment variables.....10

8. Troubleshooting.....11

9. Updating later.....11

Note: this table is generated from automatic fields. After editing the document, right-click it and choose Update Field to refresh the page numbers.

1. Overview and installation requirements

Molecular Studio is a desktop graphical workspace for the GROMACS molecular dynamics engine. It does not replace or modify GROMACS in any way; it drives the real engine (pdb2gmx, editconf, solvate, genion, grompp, mdrun and the rest of the 105-tool catalogue) and presents the whole workflow — project management, parameter editing, execution, monitoring, 3D structure display and result analysis — in a single window. The application is built in three layers: a Rust core that runs commands and parses files, a C++/Qt6 graphical interface, and GROMACS itself as an external process. A complete installation therefore needs three components in order: system libraries, then GROMACS, then the build toolchain (Rust and Qt6) used to compile the core and the interface.

System requirements: a recent Linux distribution (this guide was tested on Ubuntu 20.04 and newer, Debian and Kali), at least 4 GB of RAM (8 GB recommended for solvated protein systems) and 3 GB of free disk space (building GROMACS from source alone needs about 1.5 GB during compilation). An NVIDIA GPU is entirely optional — it only accelerates large production runs; every step in this guide works without one.

The project archive should have reached you as molecular-studio-v0.3.zip. Unpack it into a working directory before running the commands below; every path in this guide is relative to the project folder that appears after extraction:

```
unzip molecular-studio-v0.3.zip -d molecular-studio
cd molecular-studio
```

2. Install the base system packages

Before anything else, install the build tools and the graphics libraries Qt6 depends on at runtime: OpenGL and EGL development files, unzip, and git if you want to pull updates later. Open a terminal and run the following once; its effect lasts for the life of the system:

```
sudo apt update
sudo apt install -y build-essential cmake git curl unzip python3 \
    libgl1-mesa-dev libegl1-mesa-dev libegl-mesa0 libxkbcommon0
```

The build-essential package provides the g++ compiler toolchain, cmake generates the build files for the interface, and the mesa packages provide the 3D graphics libraries the molecular viewer uses on machines without a dedicated GPU. On a distribution other than Ubuntu or Debian the equivalents are: 'dnf groupinstall Development Tools' plus mesa-devel on Fedora, or 'pacman -S base-devel mesa' on Arch.

3. Install GROMACS

Three routes are available — pick exactly one. The first is the fastest and sufficient for learning and ordinary research; the second suits an existing conda workflow; the third gives the newest official release (about 20–40 minutes of compilation depending on the CPU).

3.1 Route one: from the distribution repositories (recommended)

```
sudo apt install -y gromacs gromacs-data
gmx --version
```

The repository version may be slightly older than the newest release, but every tool in the Molecular Studio catalogue is compatible with GROMACS 2021 and later.

3.2 Route two: through Conda

```
conda install -c conda-forge gromacs
gmx --version
```

Prefer this route if you keep separate conda environments per project: GROMACS and its libraries stay confined to the active environment.

3.3 Route three: build from source (newest release)

Download the official release, build it locally with its bundled FFTW, install under /usr/local/gromacs and activate its environment through GMXRC:

```
wget https://ftp.gromacs.org/gromacs/gromacs-2025.2.tar.gz
tar xf gromacs-2025.2.tar.gz
cd gromacs-2025.2 && mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/usr/local/gromacs
-DGMX_BUILD_OWN_FFTW=ON -DGMX_GPU=OFF
make -j$(nproc)
sudo make install
source /usr/local/gromacs/bin/GMXRC
gmx --version
```

If an NVIDIA driver is already installed, add -DGMX_GPU=CUDA to enable GPU acceleration in production runs.

After any of the three routes, verify the terminal can resolve gmx before continuing. The application auto-detects the engine path at start-up; if detection fails, point it there manually with the GMX_BIN variable described in section 7.

4. Install the build toolchain: Rust, then Qt6

4.1 Install Rust (builds the application core)

The core is written in Rust and built with cargo. The official installer is the rustup script, which places everything in your home directory without root privileges:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
rustc --version && cargo --version
```

If Rust is already installed, refresh it with 'rustup update' before building; the core targets the current stable release and needs no nightly features.

4.2 Install Qt6 (builds the graphical interface)

The interface needs the Qt6 Widgets, OpenGL and OpenGLWidgets components. On Ubuntu 22.04 and newer, one development package from the repositories covers all of them:

```
sudo apt install -y qt6-base-dev qt6-base-dev-tools
```

If your distribution does not ship Qt 6, or you want an exact version matching your development environment, use the aqtinstall downloader:

```
pip install aqtinstall
aqt install-qt linux desktop 6.8.3 linux_gcc_64 --outputdir $HOME/Qt
```

After aqt, the library and headers live under $HOME/Qt/6.8.3/gcc_64 — pass that path as CMAKE_PREFIX_PATH in section 5.2.

5. Build the project

5.1 Build the Rust core

Build the core first — the interface links against its shared library at build time. The resulting library lands in the core crate's target/release folder:

```
cd molecular-studio/core/rust/molecular_core
cargo build --release
ls target/release/libmolecular_core.so
```

The first build takes a minute or two; later builds are much faster.

5.2 Build the Qt interface

Return to the top project folder and generate the build files with cmake, passing two decisive paths: the Qt location in CMAKE_PREFIX_PATH and the core library in MS_CORE_LIB. Omitting the second does not stop the build, but the application then runs in a degraded mode with parsing and execution disabled, so never leave it empty. Important: never reuse a build/ folder copied from another machine or from a previous location — always delete it and configure from scratch:

```
cd molecular-studio
rm -rf build
cmake -S . -B build \
   -DCMAKE_PREFIX_PATH=$HOME/Qt/6.8.3/gcc_64 \

-DMS_CORE_LIB=$PWD/core/rust/molecular_core/target/release/libmolecular_core.so
\
   -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
```

If you installed Qt from the apt repositories, replace the CMAKE_PREFIX_PATH value with the system default or drop the option entirely — cmake will locate Qt6 automatically. On success the molecular-studio executable appears inside the build folder (about half a megabyte).

6. Run the application and verify the installation

Launch the interface from the build folder. The main window opens with the project tree on the left, the parameter editor on the right, the 3D viewer with its playback bar in the centre, and the task monitor and log along the bottom:

```
./build/molecular-studio
```

For a non-interactive smoke test after every rebuild, the following command renders the main window to an image and exits:

```
./build/molecular-studio --screenshot /tmp/test.png --shot-delay 1500
```

Then verify the four key points in order: the status bar reads Engine linked (never 'engine not linked'); Simulation → New simulation setup opens the wizard with the four system templates; the Analysis dashboard discovers your project's data files after a completed run; and Help → Load sample structure plays the bundled lysozyme in the 3D viewer. For a first real project: File → New Project, then Simulation → New simulation setup, choose the Protein in water template, and run the full pipeline — the steps execute strictly one after another, each checking its required files before it starts.

7. Important environment variables

Four variables control the application and its build chain; add them to your .bashrc to make them permanent, or type them into the current terminal for a one-off session:

```
export GMX_BIN=/usr/local/gromacs/bin/gmx
export LD_LIBRARY_PATH=/usr/local/gromacs/lib:$LD_LIBRARY_PATH
export CMAKE_PREFIX_PATH=$HOME/Qt/6.8.3/gcc_64
export MS_CORE_LIB=$HOME/molecular-studio/core/rust/molecular_core/target/
release/libmolecular_core.so
```

Table 1: what each variable does and when you need it

Variable	Purpose	When you need it
GMX_BIN	Points the core at the engine executable instead of the usual automatic search	GROMACS installed in a non-standard location, or several versions on one machine
LD_LIBRARY_PATH	Search path for the engine's shared libraries (libgromacs.so)	The error 'libgromacs.so.10: cannot open shared object file' when running gmx

Variable	Purpose	When you need it
CMAKE_PREFIX_PATH	Qt6 location for cmake at build time	The error 'Could not find a package configuration file provided by Qt6'
MS_CORE_LIB	Core library the interface links against at build time	The status bar reads 'Engine not linked' after launching the application

8. Troubleshooting

Installation failures with this kind of project fall into two families: environment settings that were missing from a terminal session, and library locations that differ between distributions. The table below covers the messages you will actually meet, with the direct fix:

Table 2: common installation errors and their fixes

Message or symptom	Cause	Fix
gmx: command not found	GROMACS not installed, or outside PATH	Reinstall via section 3, or run: source /usr/local/gromacs/bin/GMXRC
libgromacs.so.10: cannot open shared object file	Engine shared libraries outside the search path	Add the lib folder to LD_LIBRARY_PATH (Table 1), or activate GMXRC in .bashrc
Could not find Qt6 (Widgets)	cmake found no Qt6 development files	Install qt6-base-dev, or pass the aqt path via CMAKE_PREFIX_PATH
Could NOT find OpenGL / WrapOpenGL	GL or EGL development files missing	sudo apt install libgl1-mesa-dev libegl1-mesa-dev
cargo: command not found	rustup incomplete or not activated	source $HOME/.cargo/env, or reinstall Rust (section 4.1)
Status bar reads 'Engine not linked'	Interface built without MS_CORE_LIB	Re-run cmake with the libmolecular_core.so path and rebuild
CMakeCache.txt directory is different / source does not match	A build/ folder was moved between machines or kept from an old source tree	Delete the build folder completely and re-run cmake from scratch — never move build/ between locations
3D viewer black or empty	OpenGL acceleration unavailable (server or virtual machine)	Install the full mesa stack; on headless servers use: LIBGL_ALWAYS_SOFTWARE=1 ./build/molecular-studio

General rule: any 'undefined reference' message during linking means stale libraries are mixed in — delete the whole build folder and re-run cmake from scratch.

9. Updating later

After any code change, or to move to a newer project release, only the changed layer needs rebuilding: the Rust core alone if the core folder changed, or the interface alone if the src folder changed. GROMACS and Qt never need reinstalling — they are stable, system-wide installations:

```
cd molecular-studio/core/rust/molecular_core && cargo build --release
cmake --build build -j$(nproc)
```

To share the application with colleagues on the same machine or a shared server, link the executable into a folder on the common PATH: sudo cp build/molecular-studio /usr/local/bin/. The application finds its core at runtime when MS_CORE_LIB is set in each user's environment or the library path stays fixed.